

```

1  '''
2      semantic-RAG
3  START → POLICY → ROUTER → SEARCH → VALIDATE → END
4      |           SQL           --retry
5      |           --fallback
6      block= end
7
8  State
9  → Node functions
10 → Routing functions
11 → Add nodes
12 → Add edges
13 → Add conditional edges
14 → Compile
15 → Invoke
16
17 '''
18
19 from typing import TypedDict
20 from langgraph.graph import StateGraph, START, END
21 '''
22 Question: query
23 Decisions: policy_action, route
24 Results: context, answer
25 Controls: validation_status, retry_count
26 '''
27 class GraphState(TypedDict, total = False):
28     query:str
29     policy_action:str
30     route:str
31     context:str
32     answer:str
33     validation_state:str
34     retry_count:int
35
36     #{"query": "What is the borrower's current rating?", "retry_count": 0}
37     #receives state → performs one job → returns state updates
38     ##### NODE FUNCTIONS #####
39     '''
40     PolicyCheck
41     '''
42     def policy_check(state: GraphState) -> dict:
43         query = state['query'].Lower()
44         blocked_terms = ["hack", "password", "confidential"]
45         action = "BLOCK" if any( term in query for term in blocked_terms) else "ALLOW"
46         return {"policy_action":action}
47
48     def blocked_response(state:GraphState)->dict:
49         return { "answer": "This request cannot be processed"}
50
51     '''
52     RouteQuery
53     '''
54     def route_query(state:GraphState)->dict:
55         query = state['query'].Lower()
56         sql_terms = ["count", "total", "average", "how many"]
57         route = "SQL" if any(term in query for term in sql_terms) else "SEMANTIC"
58         return {"route":route}
59
60     '''
61     RAG Search
62     '''
63     def rag_search(state:GraphState)->dict:
64         query = state['query'].Lower()
65         context = f"Semantic seach result for: {query}"

```

```

66         answer = f"Rag answer generated from :{context}"
67
68     return {
69         "context": context,
70         "answer": answer,
71     }
72 '''
73 SQL Search
74 '''
75 def sql_search(state: GraphState) -> dict:
76     query = state['query'].Lower()
77     context = f"SQL search result for: {query}"
78     answer = f"SQL answer generated from :{context}"
79
80     return {
81         "context": context,
82         "answer": answer,
83     }
84
85 '''
86 VALIDATE
87 '''
88 def fallback(state: GraphState) -> dict:
89     return {
90         "answer": "I could not produce a reliable answer from the available information."
91     }
92
93 ##### ADD ROUTING FUNCTIONS #####
94 def route_after_policy(state: GraphState) -> str:
95     if state["policy_action"] == "BLOCK":
96         return "blocked_response"
97     return "route_query"
98
99 def route_after_query(state: GraphState) -> str:
100     if state["route"] == "SQL":
101         return "sql_search"
102     return "rag_search"
103
104 def route_after_validation(state: GraphState) -> str:
105     status = state["validation_state"]
106     route = state['route'].Lower()
107     if status == "PASS":
108         return END
109     if status == "FAIL":
110         return "fallback"
111     if route == "sql":
112         return "sql_search"
113     return "rag_search"
114
115 ##### ADD NODES #####
116 workflow = StateGraph(GraphState)
117 workflow.add("policy_check", policy_check)
118 workflow.add("blocked_response", blocked_response)
119 workflow.add("route_query", route_query)
120 workflow.add("rag_search", rag_search)
121 workflow.add("sql_search", sql_search)
122 workflow.add("validate", validate)
123 workflow.add("fallback", fallback)
124
125 ##### ADD CONDITIONAL EDGES #####
126 '''
127 Policy validation → BLOCK or ALLOW
128 Query routing      → RAG or SQL
129 Answer validation → PASS, RETRY, or FAIL
130 '''

```

```
131 workflow.add_edge(START,"policy_check")
132 workflow.add_conditional_edges("policy_check","route_after_policy")
133 workflow.add_conditional_edges("route_query","route_after_query")
134 workflow.add_conditional_edges("route_validation","route_after_validation")
135
136 ##### ADD NORMAL EDGES #####
137 workflow.add_edge("rag_search","validate")
138 workflow.add_edge("sql_search","validate")
139 workflow.add_edge("fallback",END)
140
141 ##### COMPILE and INVOKE #####
142 graph = workflow.compile()
143 result = graph.invoke({
144     "query":"whats the borrowers' current rating",
145     "retry_count":0
146 })
147
```